

ARES Protocol — Quick Start

How a robot connects to ARES: register, heartbeat, claim, release

REST/JSON

REST/XML

A robot does exactly four things: **register** once, **heartbeat** while it runs, **claim** the space it needs, and **release** when done. Each call carries a **key** and gets back a tiny **decision + reason**. That is the whole protocol — this page is all most integrations need.

REST/JSON and REST/XML are shown here — the same four calls in two encodings. Route planning, scheduling, snapshots and the fine-level “tier-2” calls live in **PROTOCOL-advanced.typ**.

1 Connect

Transport	Address
REST / JSON	<code>http://<host>:8080/ares/v1/...</code> — the default; send <code>application/json</code> (or nothing at all).
REST / XML	<code>http://<host>:8080/ares/v1/...</code> — send/accept <code>application/xml</code> .

Check the server is up (no key needed):

```
curl http://<host>:8080/ares/v1/health
# {"status":"ok","version":"0.1.0"}
```

That is already JSON — the read endpoints answer JSON unless you ask for XML with `Accept: application/xml`.

2 The key

Each call may carry a **key**. Four forms:

Key	Meaning
an integer, e.g. <code>1234</code>	a simple numeric password
<code>pass:<secret></code>	a password; anything after the prefix
<code>did:key:<multibase></code>	an Ed25519 identity — proved by signature, see below
<code>tok:<token></code>	a bearer token issued after proving a <code>did:key</code>

The key is set at **register** and bound to the robot. Every later call resends it; a wrong key is rejected with reason **1** (**mismatched key**). The server stores a salted Argon2id hash, never the key itself, so a stolen state file is not a list of passwords.

did:key **never travels as a password**. Registering one binds the identity but grants nothing on its own; the robot then proves it holds the private half:

```
POST /ares/v1/auth/challenge {"robot":"7"}
# -> a 32-byte nonce (hex), valid 30s
POST /ares/v1/auth/prove {"robot":"7","did":"did:key:...","signature":"<hex>"}
# -> a bearer token, valid 1h
```

Later calls send that token as `tok:<token>` in the `key` field.

The key is optional. If you omit it the server uses a shared default password — convenient but insecure (anyone can act as a robot that skipped the key). If a robot registers **with** a key, it must keep sending that same key.

3 The reply

Every call answers with the same two fields:

- `decision` — `1` = ok / granted, `0` = denied.
- `reason` — `0` = ok and `1` = mismatched key on **every** call; other values are listed per call below.

In JSON the reply is a flat object: `{"decision":1,"reason":0}`, plus `"blocked"` on a denied claim. In XML the reply root is always `<reply>`. (On XML input the request's outer tag name is ignored — only the inner fields matter.)

4 The four calls

Each call shows the JSON body and the XML body — the same request in different clothes. The robot id may be an **integer or a UUID string**; in JSON it is written **quoted** either way (see the note at the end).

4.1 Register

Bind a robot id to a key. Do this once. Optional `<alive>` is the heartbeat interval in seconds (default 2); if no heartbeat arrives for `2×` that, the server marks the robot inactive **and auto-releases all its claims** (so a crashed robot never leaves a zone stuck).

```
POST /ares/v1/robots
{"robot":"7","key":"1234","alive":2}
{"decision":1,"reason":0}
```

```
POST /ares/v1/robots
<reg><robot>7</robot><key>1234</key><alive>2</alive></reg>
<reply><decision>1</decision><reason>0</reason></reply>
```

reason	meaning
0	registered
1	mismatched key
2	already registered
3	bad id (not an integer or UUID)
4	unsupported key scheme

4.2 Heartbeat

Liveness + where the robot is. The reply is just an ack; the server timestamps it. Send one of `zone` / `node` / `edge`. Use `zone = -1` when the robot holds no zone and its location is unknown.

```
POST /ares/v1/robots/7/heartbeat
{"key":"1234","zone":42}           // or "zone":-1 if unknown
{"decision":1,"reason":0}
```

```
POST /ares/v1/robots/7/heartbeat
<hb><key>1234</key><zone>42</zone></hb>    <!-- or <zone>-1</zone> if unknown -->
<reply><decision>1</decision><reason>0</reason></reply>
```

reason	meaning
0	ok
1	mismatched key
2	not registered

4.3 Claim

Reserve a zone (or node/edge — the type is in the path). To reserve several at once, repeat `<id>`; it is **all-or-nothing**. On denial, `<blocked>` names the id that stopped it. Two optional fields:

Field	Meaning
<code>access_mode</code>	<code>0</code> = unspecified (→ exclusive), <code>1</code> = exclusive (default), <code>2</code> = shared; <code>3</code> + reserved and rejected.
<code>lease_time</code>	seconds the claim should hold: <code>0</code> (default) = unlimited, <code>X</code> = X seconds.

How several ids are sent. JSON uses an array — `"id":[42]` for one, `"id":[42,43]` for several. XML repeats the element — `<id>42</id><id>43</id>`. Release takes a **single** id in both, never an array.

Shared zones. On a zone whose capacity is greater than 1, several robots may hold it at once with `access_mode=2` (shared), up to that capacity; once full, a further shared claim is denied with reason `3` (**capacity exceeded**). An **exclusive** claim (`access_mode=1`) on a shared zone still **conflicts** with the current holders (reason `2`).

Lease time, today. `<lease_time>` is accepted but timed wall-clock expiry is not enforced yet: only `0` (unlimited) is currently effective, and a non-zero value is accepted but the claim is **not** auto-expired on a timer today. The expiry that does work is auto-release when a robot stops heartbeating (after `2×` its `alive` interval).

```
POST /ares/v1/claims/zone
{"key":"1234","robot":"7","id":[42]}
{"decision":1,"reason":0}

// several zones, atomic, exclusive, 30-second lease — id is an ARRAY even for one
{"key":"1234","robot":"7","id":[42,43],"access_mode":1,"lease_time":30}
{"decision":0,"reason":2,"blocked":43}
```

```
POST /ares/v1/claims/zone
<claim><key>1234</key><robot>7</robot><id>42</id></claim>
<reply><decision>1</decision><reason>0</reason></reply>

<!-- several zones, atomic, exclusive, 30-second lease -->
<claim><key>1234</key><robot>7</robot><id>42</id><id>43</id>
  <access_mode>1</access_mode><lease_time>30</lease_time></claim>
<reply><decision>0</decision><reason>2</reason><blocked>43</blocked></reply>
```

reason	meaning
0	granted
1	mismatched key
2	conflict — someone holds it
3	capacity exceeded (shared zone full)
4	unknown id
5	bad request (no id, or unsupported <code>access_mode</code>)

Claiming a zone reserves everything inside it — so a zone claim and a claim on a node/edge within that zone conflict with each other.

4.4 Release

Give back what this robot holds on a resource.

```
POST /ares/v1/leases/release/zone
{"key":"1234","robot":"7","id":42}
{"decision":1,"reason":0}
```

```
POST /ares/v1/leases/release/zone
<rel><key>1234</key><robot>7</robot><id>42</id></rel>
<reply><decision>1</decision><reason>0</reason></reply>
```

reason	meaning
0	released
1	mismatched key
2	no such lease (robot holds nothing there)
3	unknown id

5 A full run

```
B=http://<host>:8080/ares/v1
json() { curl -s -X POST "$B$1" -H 'content-type: application/json' -d "$2"; echo; }

json /robots          '{"robot":"7","key":"1234"}'
json /claims/zone     '{"key":"1234","robot":"7","id":[42]}'
```

```
json /robots/7/heartbeat '{"key":"1234","zone":42}'
json /leases/release/zone '{"key":"1234","robot":"7","id":42}'
```

The same run over XML — register, claim, heartbeat, release:

```
xml() { curl -s -X POST "$B$1" -H 'content-type: application/xml' -d "$2"; echo; }

xml /robots          '<reg><robot>7</robot><key>1234</key></reg>'
xml /claims/zone     '<claim><key>1234</key><robot>7</robot><id>42</id></claim>'
xml /robots/7/heartbeat '<hb><key>1234</key><zone>42</zone></hb>'
xml /leases/release/zone '<rel><key>1234</key><robot>7</robot><id>42</id></rel>'
```

Pick one and stay with it — or don't. Both encodings are the same protocol: the same ids, the same keys, the same `decision` / `reason`. A fleet can mix them freely, one format per client, and a zone claimed over JSON blocks a robot asking over XML exactly as if they shared an encoding.

Two shape differences to remember. First, the claim `id` is an **array** in JSON (`{"id":[42]}` even for one) and a **repeated element** in XML (`<id>42</id>`), while release takes a single id in both.

Second, **quote robot and key in JSON** — `{"robot":"7","key":"1234"}`, not `{"robot":7,"key":1234}`. Those two fields accept an integer **or** a UUID, so they are read as text on every transport, and a bare number is rejected with `invalid type: integer ... expected a string or integer scalar`. Every other numeric field — `id`, `zone`, `node`, `edge`, `alive`, `access_mode`, `lease_time` — is a plain JSON number.